

(1) Report

CSED_311_Team20_Lab 01

20180806 류누리

20180285 이신범

(2) Introduction

이번 랩에서는 16-bit Arithmetic/Logic Unit (ALU)를 verilog를 사용하여 구현하는 것을 목표로 한다. 총 16가지의 기능을 구현해야 한다. 각 기능은 저마다 function code를 가지고 있으며, 주어진 function code에 따라 operand들에 알맞은 연산을 해야한다. 또, signed addition/subtraction을 할 경우, overflow를 감지해야 한다. 구현해야 하는 기능들과 function code는 아래의 그림과 같다.

ALU Specification

FuncCode	Operation	Comment	FuncCode	Operation	Comment
0000	$A + B$	Signed Addition	1000	$A \oplus B$	Bitwise XOR
0001	$A - B$	Signed Subtraction	1001	$\sim(A \oplus B)$	Bitwise XNOR
0010	A	Identity	1010	$A \ll 1$	Logical Left Shift
0011	$\sim A$	Bitwise NOT	1011	$A \gg 1$	Logical Right Shift
0100	$A \& B$	Bitwise AND	1100	$A \lll 1$	Arithmetic Left Shift
0101	$A B$	Bitwise OR	1101	$A \ggg 1$	Arithmetic Right Shift
0110	$\sim(A \& B)$	Bitwise NAND	1110	$\sim A + 1$	Two's Complement
0111	$\sim(A B)$	Bitwise NOR	1111	0	Zero

이번 랩 과제를 통해서 우리는 ALU가 구체적으로 어떻게 작동하는지에 대해서 배울 수 있을 것이다.

(3) Design

아마도, Design과 Implemetation의 차이는, 여러 개의 module을 사용했을 때,

// module 간의 연결을 어떻게 해서 효율적인 프로그램을 만들었는가? -> design

// module 각각의 code를 어떻게 해서 효율적인 프로그램을 만들었는가? -> implementation

에 적는 것 같다고 이해했다.

우리는 ALU라는 하나의 큰 module 안에 구체적인 연산을 수행하는 소 module을 넣었다. Case문을 사용하여 주어진 function code에 따라 어떠한 module을 사용할지 결정하도록 구현했다. ALU가 input으로 받아들이는 것은 두 개의 operand와 function code이며, output으로 연산 결과와 Overflow flag를 내놓는다.

(4) Implementation

우선, 전체적인 구현을 위해 `always @(FuncCode or A or B) begin .. end` 문을 사용하였다.

FuncCode or A or B 가 변경될 때마다, 새로운 연산을 수행해줘야 한다고 생각했기 때문이다.

[Signed Addition]

```
ADD: begin
    C = A + B;
    OverflowFlag = ((A[15] ^ C[15]) && ~(A[15] ^ B[15])) ;
    //~(A[15] ^ B[15]) is 1 if they have same sign and 0 otherwise
    //if MSB of the result is different with A or B (A[15] ^ C[15]) is one and zero otherwise
end
```

중요한 것은 overflow flag를 설정하는 경우이다. 두 operand들이 같은 부호를 가지고 있고, 그 합이 결과가 다른 부호를 가지는 경우 overflow가 발생한 것이다. 두 operand들의 MSB의 XNOR을 구하여 그들이 같은 부호를 가지는지 확인했다. 그리고 operand 하나와 result의 MSB가 다른지 확인했다. 두 조건이 만족되는 경우 over flow flag를 1로 설정했다.

[Signed Subtraction]

```
SUB: begin
    C = A - B;
    OverflowFlag = ((A[15] ^ C[15]) && (A[15] ^ B[15])) ;
    //(A[15] ^ B[15]) is 1 if they have other sign and 0 otherwise
    //if MSB of the result is different with A or B (A[15] ^ C[15]) is one and zero otherwise
end
```

위의 addition의 경우와 유사하다. 다른 점은 두 operand들이 서로 다른 부호를 가지고 있는지 확인하는 것뿐이다. 두 operand들의 MSB의 XOR을 구하여 그들이 다른 부호를 가지는지 확인했다. 그리고 operand 하나와 result의 MSB가 다른지 확인했다. 두 조건이 만족되는 경우 over flow flag를 1로 설정했다.

[Others]

Identity	4'b0010: C = A;
Bitwise Not	4'b0011: C = ~A;
Bitwise And	4'b0100: C = A & B;
Bitwise Or	4'b0101: C = A B;
Bitwise Nand	4'b0110: C = ~(A & B);
Bitwise Nor	4'b0111: C = ~(A B);

Bitwise Xor 4'b1000: C = A ^ B;

Bitwise Xnor 4'b1001: C = ~(A ^ B);

Shift (Logical) 4'b1010: C = A << 1;

 4'b1011: C = A >> 1;

Shift (Arithmetic) 4'b1100: C = A <<< 1;

 4'b1101: if(A>=16'h8000) C = ((A >> 1) | (1 << 15));

 else C = A >> 1;

Two's Comp 4'b1110: C = ~A + 1;

Zero 4'b1111: C = 4'b0000;

[Arithmetic Shift]

```

*/
module ALU (A, B, FuncCode, C, OverflowFlag);
    input [`NumBits-1:0] A; // 16bits A_input
    input [`NumBits-1:0] B; // 16bits B_input
    input [3:0] FuncCode; // 4bits logic
    output [`NumBits-1:0] C; // 16bits C_output
    output OverflowFlag;

```

맨 위 module 선언에서 A, B, C가 input signed[`NumBits-1:0] A; 가 아닌,

input [`NumBits-1:0] A; 로 선언이 되어있었다. (unsigned integer)

따라서, Arithmetic right Shift에서, A = 0'h8000 이라도, MSB를 '0'으로 인식하는 문제가 있었다.

이를 구현하기 위해 직접 Arithmetic Shift를 구현해주었다.

(5) Discussion

1]

처음 ModelSim을 작동시켰을 때, 수행한 결과를 관찰하는 방법을 몰라 어려움이 있었다.

Simulation을 한 뒤 Wave_form을 출력시켰을 때, 결과를 어떻게 읽어야 하는지 배우기 위해 PPT에 있었던 강의와, 추가적으로 AND_OR gate를 만들어 tb_file 을 Simulation 하는 방법을 익혔

다

alu.v	✓	Verilog	0	03/29/2020 05:34:47 ...
aoi_gate_tb.v	✓	Verilog	4	03/28/2020 06:59:36 ...
test_alu_tb.v	✓	Verilog	6	03/28/2020 07:27:05 ...
GATE_TB.v	✓	Verilog	9	03/28/2020 07:43:56 ...
alu_tb.v	✓	Verilog	1	03/24/2020 09:30:47 ...
GATE_AND.v	✓	Verilog	7	03/28/2020 07:46:30 ...
alu_tb_f.v	✓	Verilog	2	03/24/2020 09:30:49 ...
aoi_gate.v	✓	Verilog	3	03/28/2020 06:51:10 ...
test_alu.v	✓	Verilog	5	03/28/2020 07:27:04 ...
GATE.v	✓	Verilog	8	03/28/2020 07:47:05 ...

2]

Arithmetic Shift를 $A \gg 1$ 로 구현해주었는데, fail 값이 떠서 무엇이 틀렸는지 고민을 좀 했다.

인터넷에 검색해보니, A,B,C 가 애초에 Signed integer가 아니라 Unsigned integer로 정의되어있는 것을 확인하였다. Base code를 건드리지 않고 원하는 결과를 얻기 위해, Arithmetic Shift를 직접 구현시켜주었다.

3]

첫 Report라서 그런지, Design 과 Implementation 항목의 차이를 잘 모르겠다.

Design은 Module 과 Module 간의 연결을 어떻게 시켰는지 작성하고,

Implementation은 각 Module 내부에서 어떻게 연산이 진행되는지를 설명하면 되는 것인가?

(6) Conclusion

1] Did you succeed in achieving the goals described in the introduction section?

모든 case를 성공했다. Wave form Simulation결과 fail은 단 한 개도 나오지 않았다.

2] If not, which goals could not be achieved? Why?

모든 case를 성공했다.